UD02 – Actividades voluntarias y obligatoria:

Introducción a las bases de datos



Autor: Óscar García Escudero
Asignatura: ADA

Índice

Introducción....4
Objetivos....4
Desarrollo de las actividades....5
Actividad UD01....5
Resultado....6
Actividad UD02....6
Resultado....8
Actividad UD03....9
Resultado....14
Actividad UD04....16
Resultado....27
Actividad Obligatoria....28
Resultado....37
Problemas encontrados, soluciones y/o sugerencias....38
Conclusión....39
Bibliografía/webgrafía....39

Licencia



Reconeixement - No comercial – CompartirIgual(BY-NC-SA): No es permet un ús comercial de l'obra original ni de les possibles obres derivades, la distribució de les quals s'ha de fer amb una llicència igual a la que regula l'obra original.

Introducción

En esta unidad didáctica se va a realizar un repaso a las bases de datos teniendo en cuenta la necesidad de almacenar de forma persistente la información que pueden manejar para poder utilizarla en otro momento. En concreto para esta unidad, se va a trabajar conceptos relacionados con las bases de datos relacionales a través de tablas haciendo uso del SGBD SQLite integrada en Python a través de la librería sqlite3.

Objetivos

- Conocer los comandos más habituales y el uso del terminal de SQLite para trabajar con las tablas de una base de datos.
- Repasar el uso de los lenguajes DDL, DML y DQL a través del terminal SQLite.
- Aprender a utilizar los lenguajes anteriores en Python con la librería sqlite3

Desarrollo de las actividades

Repositorio de GitHub compartido: https://github.com/oscgaresc/ADA

Actividad UD01

Diseña una sentencia SQL que cree la tabla LISTA_HERRAMIENTAS utilizando el fichero de bases de datos ejers_voluntarios_ejer1.db. No es necesario que escribas un programa de Python. Simplemente crea la sentencia y ejecútala para ver que funciona. Deberás tener en cuenta los siguientes puntos:

- ***Los dos campos son de tipo TEXT.***
- ***La clave primaria de la tabla son los dos campos que contiene.***
- ***El campo jugador referencia al campo nombre de la tabla JUGADORES.***
- ***El campo herramienta referencia al campo nombre de la tabla HERRAMIENTAS.***

Cuando hayas acabado, ejecuta el comando .schema LISTA_HERRAMIENTAS y sube una captura de pantalla al aula virtual.

Siguiendo las indicaciones del ejercicio se crea el fichero .sql con el siguiente código:

```
#######################################
# UNIDAD 2 - Ejercicio 1              #
# crear la tabla LISTA_HERRAMIENTAS   #
#######################################
# sqlite3
# .open ejers_voluntarios_ejer1.db
# .read ud2_ejer1_tabla_lista_herramientas.sql

.mode column
# las tablas JUGADORES y HERRAMIENTAS ya están creadas
# CREATE TABLE IF NOT EXISTS JUGADORES (
#   nombre TEXT,
#    PRIMARY KEY(nombre)
# );

# CREATE TABLE IF NOT EXISTS HERRMIENTAS(
#    nombre TEXT,
#    PRIMARY KEY(nombre)
# )

CREATE TABLE IF NOT EXISTS LISTA_HERRAMIENTAS (
  jugador TEXT,
```

```
  herramienta TEXT,
  PRIMARY KEY(jugador, herramienta),
  FOREIGN KEY(jugador) REFERENCES JUGADORES(nombre),
  FOREIGN KEY(herramienta) REFERENCES HERRAMIENTAS(nombre)
);
```

Resultado

```
sqlite> .read ..\Ejer1Voluntario\ud2_ejer1_tabla_lista_herramientas.sql
sqlite> .schema LISTA_HERRAMIENTAS
CREATE TABLE LISTA_HERRAMIENTAS (
    jugador TEXT,
    herramienta TEXT,
    PRIMARY KEY(jugador, herramienta),
    FOREIGN KEY(jugador) REFERENCES JUGADORES(nombre),
    FOREIGN KEY(herramienta) REFERENCES HERRAMIENTAS(nombre)
);
sqlite> .tables
HERRAMIENTAS     JUGADORES     LISTA_HERRAMIENTAS
```

Actividad UD02

Crea un programa llamado ud2_ejer2_poblar_tabla.py que se encargará de insertar distintos registros en la tabla LISTA_HERRAMIENTAS.

Recuerda que deberás crear una conexión a la base de datos ejers_voluntarios_ejer2.db y utilizar un cursor para ejecutar las instrucciones.

Ten en cuenta los siguientes puntos:

- ***Crea el siguiente registro utilizando la función execute() sin parámetros.***

```
jugador  herramienta
-------  -----------
ana      alas
```

- ***Crea el siguiente registro utilizando la función execute() pasando como parámetro el valor de los dos campos.***

```
jugador  herramienta
-------  -----------
ana      botines
```

- ***Crea los siguientes registros utilizando la función executemany() y pasando la lista de tuplas correspondiente.***

```
jugador  herramienta
-------  -----------
ana      rastrillo
pep      alas
pep      varita
mary     varita
mary     esencia
mary     martillo
quico    botines
quico    rastrilo
sandy    varita
```

• ***Recuerda ejecutar la función commit() para que los cambios se hagan permanentes.***

El código final del script donde se han seguido las indicaciones de los ejercicios de repaso:

```
# importar libreria del SGBD sqlite
import sqlite3
import os # también para evitar problemas rutas

#se muestra la ruta de trabajo actual amb el metode getcwd() ""
print(f"Trabajando en la ruta:\n{os.getcwd()}")


ruta_fichero = input("\nIndica la ruta relativa hasta el fichero .db"
           +" (intro si es la misma): ")
# EjerciciosVoluntarios_Obligatorio\Ejer2Voluntario
nomFichero = input('\nIndica el nombre del fichero: ')
# ejers_voluntarios_ejer2.db
ruta_completa = os.path.join(os.getcwd(), ruta_fichero, nomFichero) #directorio
real de trabajo
# print(f'Ruta de trabajo actual: {ruta_completa}')

# conectando a la base de datos ejers_voluntarios_ejer2.db
my_conn = sqlite3.connect(ruta_completa)

# generando el cursos u objeto de la BD para ejecutar instrucciones SQL
my_cur = my_conn.cursor()
# sentencias/instrucciones
# insert directo
ins1 = '''insert into LISTA_HERRAMIENTAS
  values ('ana', 'alas')'''
# cuidado con las "" en los TEXT, se interpretan como atributo de la
# tabla no como dato

# insert con dos parámetros ??
ins2 = ''' INSERT INTO LISTA_HERRAMIENTAS
  VALUES(?, ?)''' # con dos parámetros y una tupla
# # insert con una lista de tuplas
```

```python
lista = [('ana', 'rastrillo'),
    ('pep', 'alas'),
    ('pep', 'varita'),
    ('mary', 'varita'),
    ('mary', 'esencia'),
    ('mary', 'martillo'),
    ('quico', 'botines'),
    ('quico', 'rastrilo'),
    ('sandy', 'varita')]

# utilizando execute()
#sin parámetros
my_cur.execute(ins1) #insertando una linea de datos
# con dos parámetros
my_cur.execute(ins2,('ana','botines')) # hay que añadir los parámetros en una tupla
# # con una lista de tuplas
my_cur.executemany(ins2,lista)


my_conn.commit() # un commit por transacción

# cerrando la conexión a la BD
my_conn.close()

print("\n¡Tabla verificada e inserciones realizadas con éxito!")
```

Para la realización del ejercicio hay que tener en cuenta el directorio de trabajo ya que puede dar problemas, de ahí que se le solicite al usuario la ruta del fichero de la base de datos para evitar que no encuentre la base de datos.

Resultado

Siguiendo las indicaciones del ejercicio se realizas las pruebas para comprobar su funcionamiento:

- Paso 0: la ejecución del script:

```
ntarios_Obligatorio/Ejer2Voluntario/ud2_ejer2_poblar_tabla.py
Trabajando en la ruta:
E:\USB\IES_SerraPerenxisa\Curso2\Curso2026\ADA\UD02Python_SQL

Indica la ruta relativa hasta el fichero .db (intro si es la misma): EjerciciosVoluntarios_Obligatorio\Eje
r2Voluntario

indica el nombre del fichero: ejers_voluntarios_ejer2.db

¡Tabla verificada e inserciones realizadas con éxito!
```

- Paso 1: probando el script inical

```
sqlite> select * from LISTA_HERRAMIENTAS;
```

jugador	herramienta
ana	alas

- Paso 1, 2 y 3 con el script completo:

```
sqlite> select * from LISTA_HERRAMIENTAS;
```

jugador	herramienta
ana	alas
ana	botines
ana	rastrillo
pep	alas
pep	varita
mary	varita
mary	esencia
mary	martillo
quico	botines
quico	rastrilo
sandy	varita

En este último caso se observa como al ejecutar el script de python con todas las sentencias insert donde se comprueba los cambios permanentes en la base de datos

Actividad UD03

Vamos a crear un programa que permita consultar la información almacenada en la base de datos ejers_voluntarios_ejer3.db. Al principio del programa habrá que crear la conexión contra la base de datos y el cursor que se va a utilizar para manejar las instrucciones SQL.

El programa mostrará el siguiente menú:

```
========================================
         CONSULTAR BASE DE DATOS
========================================
 1 - Imprimir jugadores
 2 - Imprimir herramientas-jugador
 3 - Consultar jugador
 4 - Consultar herramienta-jugador
 0 - SALIR
------------------------
Dame la opción:
```

Cada opción seleccionada ejecutará una nueva función que responderá a la tarea.

Opción	Función

1	***imprimir_jugadores()***
	Esta función leerá todos los registros de la tabla JUGADORES y mostrará la información por pantalla.
	Ejemplo:
	`IMPRIMIR JUGADORES`
	`Nombre: ana - Vida: 100`
	`Nombre: pep - Vida: 120`
	`Nombre: mary - Vida: 80`
	`Nombre: quico - Vida: 100`
	`Nombre: sandy - Vida: 80`
	A la función se le pasará como parámetro de entrada el cursor a la base de datos y no devolverá ningún parámetro de salida.

2	***imprimir_herramientas_jugador()***
	Esta función pedirá el nombre de un jugador y mostrará por pantalla todas las herramientas que tiene asociadas. Para ello, consultará la tabla LISTA_HERRAMIENTAS.
	Ejemplo:
	`HERRAMIENTAS JUGADOR`
	`Dame el nombre del jugador: ana`
	`Herramientas: alas - botines - rastrillo -`
	A la función se le pasará como parámetro de entrada el cursor a la base de datos y no devolverá ningún parámetro de salida.

3	***consultar_jugador()***
	Esta función pedirá por pantalla el nombre del jugador y mostrará sus datos. Para ello, se consultará la tabla JUGADORES.
	Ejemplo:
	`CONSULTAR JUGADOR`
	`Dame el nombre del jugador: ana`
	`Nombre: ana - Vida: 100`
	A la función se le pasará como parámetro de entrada el cursor a la base de datos y no devolverá ningún parámetro de salida

4	***consultar_herramienta_jugador()*** Esta función pedirá por pantalla el nombre de un jugador y de una herramienta y comprobará si dicho jugador tiene asociada dicha herramienta. Para ello, se consultará la tabla LISTA_HERRAMIENTAS. Ejemplo 1: El jugador sí tiene asociada la herramienta.

	CONSULTAR HERRAMIENTA-JUGADOR Dame el nombre del jugador: ana Dame herramienta: botines El jugador <ana> sí tiene la herramienta <botines> Ejemplo 2: El jugador no tiene asociada la herramienta. CONSULTAR HERRAMIENTA-JUGADOR Dame el nombre del jugador: ana Dame herramienta: varita El jugador <ana> no tiene la herramienta <varita> A la función se le pasará como parámetro de entrada el cursor a la base de datos y no devolverá ningún parámetro de salida.
0	El programa acaba.

Cuando se acabe de ejecutar una acción, el programa volverá a mostrar el menú al usuario, excepto si se ha seleccionado la opción 0.

Puedes crear el programa desde cero o basarte en el esquema que tienes en el fichero ud2_ejer3_consultar_bd (ESQUEMA).py del aula virtual. En este caso, deberás cambiar las instrucciones pass que encuentres en el código.

A continuación se muestra el código teniendo en cuenta la descripción de las funciones y que no se ha mirado en ningún momento las soluciones, por tanto, aunque las pruebas han salido correctas pueden aparecer errores involuntarios:

```
def imprimir_jugadores(cursor):
  '''Imprimir jugadores existentes en la base de datos.

  Args
  ---
  cursor (cursor)
    Objeto de la BDA que permite ejecutar instrucciones SQL y recuperar registros
de la BDA.

  Returns:
    Out: No hay. Muestra por consola las columnas nombre y vida de la tabla
JUGADORES de la BDA.
  '''
  print("IMPRIMIR JUGADORES")
  cursor.execute('SELECT nombre, vida from JUGADORES')
  # fechone() / fetchall() para recuperar los resultados
  reg = cursor.fetchone() # retorna la tupla con los valores de las columnas
  c = 0 # contador de jugadores
  while reg != None: # None si no hay nada que retornar
    c+=1
    print(f'jugador {c:>2}: Nombre: {reg[0]:<6} - Vidas: {reg[1]:>3} ')
    reg = cursor.fetchone() # siguiente registro
```

```
def imprimir_herramientas_jugador(cursor):
  '''Imprimir las herramientas de un jugador

  Args
  ---
   cursor (cursor)
    Objeto de la BDA que permite ejercutar instrucciones SQL y recuperar registros
de la BDA.

  Returns:
    No hay. Muestra por consola las herramientas de un jugador seleccionado por el
usuario.
  '''
  print("HERRAMIENTAS JUGADOR")
  cursor.execute('SELECT * from LISTA_HERRAMIENTAS')
    # fechone() / fetchall() para recuperar los resultados
  registros = cursor.fetchall() # retorna tupla con los valores de todas las
columnas
  nombre = input ('Para conocer las herramientas de un jugador, indica su nombre:
')
  estaRegistrado = False # centinela
  herramientas = [] # lista de herramientas vacía
  for reg in registros:
    if reg[0]==nombre.lower(): # Compara la columna 0, es decir, la columna nombre
de la tabla
      herramientas.append(reg[1]) # se añade la herramienta de la columna 1 a la
lista
      estaRegistrado = True
  if not estaRegistrado:
    print(f'\u001B[33mWARNING: El jugador {nombre} no se encuentra'
       +f' en la base de datos.\u001B[0m')
  else:
    print(f'El/la jugador/a {nombre} tiene las herramientas:', end=" ")
    for elem in herramientas:
       print(f'{elem}', end=' - ')
  print()

def consultar_jugador(cursor):
  '''Comprueba si existe un jugador en la tabla JUGADORES.
  Args
  ---
   cursor (cursor)
    Objeto de la BDA que permite ejercutar instrucciones SQL y recuperar registros
de la BDA.
```

```
  Returns:
    No hay. Muestra por consola las vidas de un jugador seleccionado por el
usuario.
  '''
  print("CONSULTAR JUGADOR")
  cursor.execute('SELECT * from JUGADORES')
      # fechone() / fetchall() para recuperar los resultados
  registros = cursor.fetchall() # retorna tupla con los valores de las columnas
  nombre = input ('Para conocer las vidas de un jugador, indica su nombre: ')
  estaRegistrado = False # centinela
  vida = ''
  for reg in registros:
    if reg[0]==nombre.lower():
      vida = reg[1]
      estaRegistrado = True
  if not estaRegistrado:
    print(f'\u001B[33mWarning: El jugador {nombre} no se encuentra'
       +f' en la base de datos.\u001B[0m')
  else:
    print(f'El/la jugador/a {nombre} tiene una vida de: {vida}')

def consultar_herramienta_jugador(cursor):
  '''Comprueba si existe el par jugador-herramienta.

  Args
  ---
  cursor (cursor)
    Objeto de la BDA que permite ejercutar instrucciones SQL y recuperar registros
de la BDA.

  Returns:
    No hay. Muestra por consola la relación entre un jugador seleccionado por el
usuario y sus herramientas.
  '''
  print("CONSULTAR HERRAMIENTA-JUGADOR")
  cursor.execute('SELECT * from LISTA_HERRAMIENTAS')
  # fechone() / fetchall() para recuperar los resultados
  registros = cursor.fetchall() # retorna tupla con los valores de las columnas
  nombre = input ('Indica el nombre del jugador: ')
  estaRegistrado = False # centinela
  herramientas = [] # lista de herramientas vacía
  for reg in registros:
    if reg[0]==nombre.lower():
      herramientas.append(reg[1])
      estaRegistrado = True
  if len(herramientas) == 0 or not estaRegistrado:
```

```
    print(f'\u001B[33mWarning: El jugador {nombre} no se encuentra'
          +f' en la base de datos o no tiene herramientas.\u001B[0m')
  else:
    eina = input ('Indica el nombre de la herramienta: ')
    for h in herramientas:
      #ternaria
      if h == eina:
        estaRegistrado = True
        break
      else: estaRegistrado = False

    if estaRegistrado:
      print(f'El/la jugador/a {nombre} tiene la herramienta: {eina}')
    else:
      print(f'\u001B[33mWarning: El/la jugador/a {nombre} No tiene'
         +f' la herramienta: {eina}\u001B[0m')

# PROGRAMA PRINCIPAL ------------------------------------------
#se muestra la ruta de trabajo actual amb el metode getcwd() ""
print(f"Trabajando en la ruta:\n{os.getcwd()}")
ruta_fichero = input("\nIndica la ruta relativa hasta el fichero .db"
          +" (intro si es la misma): ")
# EjerciciosVoluntarios_Obligatorio\Ejer2Voluntario
nomFichero = input('\nIndica el nombre del fichero de la BD: ')
# ejers_voluntarios_ejer2.db
ruta_completa = os.path.join(os.getcwd(), ruta_fichero, nomFichero) #directorio
real de trabajo
print (f'Ruta a conectar:n{ruta_completa}')
conexion = sqlite3.connect(ruta_completa)
#Se abre la conexión con la base de datos
cursor = conexion.cursor()

opcion = valida_opcion() . . .
while opcion != '0':
.
.
.
#Se cierra la conexión con la base de datos.
conexion.close()
```

Resultado

El resultado de las pruebas es el siguiente:



Como se puede observar en la imagen anterior, al inicio solicita la ruta y el fichero de la base de datos, si está contenida en el mismo directorio de trabajo solo hay que realizar un *intro* y añadir el fichero de base de datos. En caso contrario, añadir la dirección relativa hasta llegar al fichero .db.

A partir de aquí, se realizan las posibles pruebas teniendo en cuenta las funciones descritas en esta actividad donde se puede ver en cada uno de los casos, a partir de los datos añadidos por el usuario.

Se han realizado cambios en relación a la última defensa mejorando la documentación de las funciones, añadiendo alertas *Warning*, organizando mejor el contenido al imprimir por consola, se ha mejorado la experiencia de usuario (UX) a la hora de notificar errores cuando toca, entre otras.

Actividad UD04

Vamos a crear un programa que permita consultar la información almacenada en la base de datos ejers_voluntarios_ejer4.db. Al principio del programa habrá que crear la conexión contra la base de datos y el cursor que se va a utilizar para manejar las instrucciones SQL.

El programa mostrará el siguiente menú:

```
========================================
         GESTIONANDO JUGADORES
========================================
 1 - Imprimir jugadores
 2 - Modificar vida de jugador
 3 - Insertar nuevo jugador
 4 - Imprimir herramientas-jugador
 5 - Quitar herramienta-jugador
 0 - SALIR
-------------------------
Dame la opción:
```

Cada opción seleccionada ejecutará una nueva función que responderá a la tarea.

Opción	Función

1	***imprimir_jugadores()***
	Esta función leerá todos los registros de la tabla JUGADORES y mostrará la información por pantalla.
	Ejemplo:
	`IMPRIMIR JUGADORES`
	`Nombre: ana - Vida: 100`
	`Nombre: pep - Vida: 120`
	`Nombre: mary - Vida: 80`
	`Nombre: quico - Vida: 100`
	`Nombre: sandy - Vida: 80`
	A la función se le pasará como parámetro de entrada el cursor a la base de datos y no devolverá ningún parámetro de salida.

2	***modificar_vida()*** Esta función pedirá por pantalla el nombre del jugador y la vida (número entero) y actualizará el campo VIDA de la tabla JUGADORES para aquel registro cuyo campo NOMBRE coincida con el introducido por pantalla.

<table>
<tr><td></td><td>Ejemplo 1: El jugador existe en la base de datos.

```
MODIFICAR VIDA
Dame jugador: ana
Dame vida: 200
La vida del jugador <ana> se ha modificado correctamente
```

```
nombre  vida
------  ----
ana     200
```

La función deberá comprobar que el jugador existe en la base de datos antes de intentar modificarlo y, si no es así, mostrar un mensaje de error. Para ello se creará un función nueva llamada existe_jugador() que se explica más adelante.

Ejemplo 2: El jugador no existe en la base de datos.

```
MODIFICAR VIDA
Dame jugador: kk
ERROR: El jugador no existe
```

A la función se le pasará como parámetro de entrada la conexión a la base de datos y el cursor pero no devolverá ningún parámetro de salida.</td></tr>
<tr><td>3</td><td>insertar_jugador()

Esta función leerá los elementos de la lista del programa y, cada elemento, lo almacenará en una línea distinta del fichero. Deberás añadir el carácter de fin de línea (“\n”) antes de escribir cada línea en el fichero.

Ejemplo 1: Añadimos un nuevo jugador a la base de datos.

```
INSERTAR JUGADOR
Dame nombre: perico
Dame vida: 200
El jugador <perico> se ha insertado correctamente
```

```
nombre  vida
------  ----
perico  200
```

La función deberá comprobar que el jugador no existe en la base de datos antes de intentar modificarlo y, si no es así, mostrar un mensaje de error. Para ello se creará un función nueva llamada existe_jugador() que se explica más adelante.

Ejemplo 2: El jugador ya existe en la base de datos.

```
INSERTAR JUGADOR
Dame nombre: ana
ERROR: El jugador ya existe
```

A la función se le pasará como parámetro de entrada la conexión a la base de datos y el cursor pero no devolverá ningún parámetro de salida.</td></tr>
<tr><td>4</td><td>imprimir_herramientas_jugador()

Esta función imprimirá la lista de herramientas que posee el jugador leído por pantalla. Para ello, consultará la tabla LISTA_HERRAMIENTAS de la base de datos.

Ejemplo 1: Buscamos las herramientas de un jugador existente.

```
HERRAMIENTAS JUGADOR
Dame el nombre del jugador: ana
Herramientas: alas - botines - rastrillo -
```

La función deberá comprobar que el jugador existe en la base de datos antes de</td></tr>
</table>

<table>
<tr><td></td><td>intentar modificarlo y, si no es así, mostrar un mensaje de error. Para ello se creará un función nueva llamada existe_jugador() que se explica más adelante.

Ejemplo 2: El jugador no existe en la base de datos.

HERRAMIENTAS JUGADOR
Dame el nombre del jugador: kk
ERROR: El jugador no existe

A la función se le pasará como parámetro de entrada el cursor pero no devolverá ningún parámetro de salida.</td></tr>
<tr><td>5</td><td>quitar_herramienta()

Esta función pedirá un nombre de jugador y un nombre de herramienta por pantalla y eliminará dicha herramienta de la lista de herramientas de dicho jugador. Para ello, utilizará la tabla LISTA_HERRAMIENTAS de la base de datos.

Ejemplo 1: Quitamos una herramienta de un jugador existente.

QUITAR HERRAMIENTA
Dame el nombre del jugador: pep
Dame herramienta: alas
Se ha eliminado la herramienta <alas> del jugador <pep> correctamente

La función deberá comprobar que el jugador existe en la base de datos antes de intentar modificarlo y, si no es así, mostrar un mensaje de error. Para ello se creará un función nueva llamada existe_jugador() que se explica más adelante.

Ejemplo 2: El jugador no existe en la base de datos.

QUITAR HERRAMIENTA
Dame el nombre del jugador: kk
ERROR: El jugador no existe

La función deberá comprobar que el jugador tiene asignada dicha herramienta y, si no es así, mostrará un mensaje de error. Para ello se creará un función nueva llamada existe_herramienta_jugador() que se explica más adelante.

Ejemplo 3: El jugador no tiene asociada la herramienta.

QUITAR HERRAMIENTA
Dame el nombre del jugador: pep
Dame herramienta: capa
ERROR: El jugador no tiene asignada la herramienta <capa>

A la función se le pasará como parámetro de entrada la conexión a la base de datos y el cursor pero no devolverá ningún parámetro de salida.</td></tr>
<tr><td>0</td><td>El programa acaba.</td></tr>
</table>

Cuando se acabe de ejecutar una acción, el programa volverá a mostrar el menú al usuario, excepto si se ha seleccionado la opción 0.

En esta tabla se describen las funciones adicionales que debes programar:

Función	Descripción
existe_jugador()	Esta función comprobará si el jugador existe en la tabla JUGADORES pero no mostrará nada por pantalla. A la función se le pasará como parámetro de entrada el cursor de la base de datos y el nombre del jugador y devolverá como parámetro de salida un valor lógico: ***True*** si el jugador existe en la tabla JUGADORES y ***False*** en otro caso.
existe_herramienta_jugador()	Esta función comprobará si en la tabla LISTA_HERRAMIENTAS hay algún registro donde se asocie la herramienta al jugador pero no mostrará nada por pantalla. A la función se le pasará como parámetro de entrada el cursor de la base de datos y el nombre del jugador y devolverá como parámetro de salida un valor lógico: ***True*** si el jugador existe en la tabla JUGADORES y ***False*** en otro caso.

En algunos casos hará falta manejar excepciones y, en otros, solo será necesario imprimir un aviso al usuario.

Puedes crear el programa desde cero o basarte en el esquema que tienes en el fichero ud2_ejer4_jugadores (ESQUEMA).py del aula virtual. En este caso, deberás cambiar las instrucciones pass que encuentres en el código.

A continuación se muestra el código teniendo en cuenta la descripción de las funciones y que no se ha mirado en ningún momento las soluciones, por tanto, aunque las pruebas han salido correctas pueden aparecer errores involuntarios. Por otro lado, el uso de excepciones se ha evitado siempre que los propios métodos de python lo han permitido:

```
# DEFINICIÓN DE FUNCIONES -------------------------------------
def valida_opcion():
  '''Función que muestra un menú y valida que la opción sea correcta'''

  opc_correctas = ['1', '2', '3', '4', '5', '0']

  #Mostramos el menú y solicitamos la operación al usuario
  opcion = ''
  while opcion not in opc_correctas:
    print()
    print("========================================")
    print("         GESTIONANDO JUGADORES          ")
    print("========================================")
    print(" 1 - Imprimir jugadores")
    print(" 2 - Modificar vida de jugador ")
    print(" 3 - Insertar nuevo jugador ")
    print(" 4 - Imprimir herramientas-jugador ")
```

```
    print(" 5 - Quitar herramienta-jugador ")
    print(" 0 - SALIR ")
    print("--------------------------")
    opcion = input("Dame la opción: ")
    if opcion not in opc_correctas:
      print("Por favor, vuelve a intentarlo.")
    else:
      print()
  return opcion

def imprimir_jugadores(cursor):
  '''Imprimir jugadores existentes en la base de datos

  Args
  ---
  cursor (cursor)
    Objeto de la BDA que permite ejecutar instrucciones SQL y recuperar registros
de la BDA.

  Returns:
    Out: No hay. Muestra por consola las columnas nombre y vida de la tabla
JUGADORES de la BDA.
  '''
  print("IMPRIMIR JUGADORES")
  cursor.execute('SELECT nombre, vida from JUGADORES')
  # fechone() / fetchall() para recuperar los resultados
  reg = cursor.fetchone() # retorna la tupla con los valores de las columnas
  c = 0 # contador de jugadores
  while reg != None: # None si no hay nada que retornar
    c+=1
    print(f'jugador {c:>2}: Nombre: {reg[0]:<6} - Vidas: {reg[1]:>3} ')
    reg = cursor.fetchone() # siguiente registro


def existe_jugador(cursor, jugador):
  '''Comprueba si existe un jugador en la tabla JUGADORES.

  Args
  ---
  cursor (cursor)
    Objeto de la BDA que permite ejecutar instrucciones SQL y recuperar registros
de la BDA.
  juegador (string)
    necesario para comparar con los jugadores de la BDA.

  Returns:
```

```
    Boolean: devuelve True si existe y False en otro caso.
  '''
  cursor.execute('SELECT * from JUGADORES where nombre = ?',(jugador,))
  # fechone() / fetchall() para recuperar los resultados
  registro = cursor.fetchone() # retorna tupla con los valores de las columnas
  return (registro != None)


def existe_herramienta_jugador(cursor, jugador, herramienta):
  '''Comprueba si existe el par jugador-herramienta.

  Args
  ---
  cursor (cursor)
    Objeto de la BDA que permite ejecutar instrucciones SQL y recuperar registros
de la BDA.
  jugador (string)
    necesario para comparar con los jugadores de la BDA.
  herramientas (string)
      necesario para comparar con las herramientas de la BDA.

  Returns:
    Boolean: devuelve True si existe y False en otro caso.
  '''
  #Buscando en la BDA el registro indicado por el usuario
  cursor.execute('SELECT * from LISTA_HERRAMIENTAS where jugador = ? AND '+
          'herramienta = ?',(jugador,herramienta))
  # fechone() / fetchall() para recuperar el resultado específico
  registros = cursor.fetchone() # retorna tupla con el único valor seleccionado
  # cuidado con fetchall() aquí
  return (registros != None)

def lee_vida():
  '''Pide el valor "vida" por pantalla.

  Args
  ---
    No hay.

  Returns:
    Integer: número de vida del jugador.
  '''
  correcto = False
  while not correcto:
    try:
      vida = int(input("Dame vida: ").strip())
```

```
    except ValueError:
      print("\u001B[33mWARNING: Se está esperando un número entero. Por favor,"
        +" vuelve a intentarlo.\u001B[0m")
    else:
      correcto = True
  return vida

def modificar_vida(conexion, cursor):
  '''Modifica la vida de un jugador

  Args
  ---
  cursor (cursor)
    Objeto de la BDA que permite ejercutar instrucciones SQL y recuperar registros
de la BDA.
  Conexion (sqlite3.connect())
    Para realizar realizar un commit al update que se realiza en la BDA.

  Returns:
    No hay. Se muestra por consola que se ha actualizado la "vida" en la BDA.
  '''
  print("MODIFICAR VIDA")
  nombre = input ('Indica el nombre del jugador: ').lower().strip()
  if not existe_jugador(cursor, nombre):
    print(f'\u001B[33mWARNING: El/la jugador/a {nombre} no existe.\u001B[0m')
    return
  vida = lee_vida() # Solicita vida al usuario

  cursor.execute('update JUGADORES set vida = ? where nombre = ?'
          ,(int(vida),nombre)) # siempre con tupla
  conexion.commit()
  print(f'\u001B[32mSe ha actualizado la vida al jugador/a {nombre}.\u001B[0m')

  # Se puede realizar directamente en update a través de los datos del usuario
  # y utilizar cursor.rowcount donde el caso de devolver un 0 quiere decir que
  # no se ha actualizado ningún jugador de la tabla JUGADORES, por tanto, el
  # nombre del jugador no existe en la base de datos.

def insertar_jugador(conexion, cursor):
  '''Inserta una nuevo jugador en la base de datos

  Args
  ---
  cursor (cursor)
    Objeto de la BDA que permite ejercutar instrucciones SQL y recuperar registros
de la BDA.
```

```
  Conexion (sqlite3.connect())
    Para realizar realizar un commit al insert que se realiza en la BDA.

  Returns:
    No hay. Se muestra por consola que se ha añadido un jugador en la BDA.
  '''
  print("INSERTAR JUGADOR")
  nombre = input ('Indica el nombre del jugador: ').lower().strip()
  if existe_jugador(cursor, nombre):
    print(f'\u001B[33mWARNING: el/la jugador/a {nombre} ya existe.\u001B[0m')
    return
  vida = lee_vida() # Solicita vida del jugador al usuario.
  ins = 'insert into JUGADORES values(?,?)'
  cursor.execute(ins,(nombre, int(vida))) # int para asegurar el entero
  conexion.commit()
  print(f'\u001B[32mSe ha insertado el jugador/a "{nombre}" con vida {vida}.
\u001B[0m')

def existe_herramienta_asociada_jugador(cursor, jugador):
  '''Comprueba si existe el una herramienta asociada al jugador en la tabla
LISTA_HERRAMIENTAS

  Args
  ---
  cursor (cursor)
    Objeto de la BDA que permite ejecutar instrucciones SQL y recuperar registros
de la BDA.
  jugador (string)
    necesario para comparar con los jugadores de la BDA.

  Returns:
    Boolean: devuelve True si existe y False en otro caso.
  '''
    #Buscando en la BDA el registro indicado por el usuario
  cursor.execute('SELECT * from LISTA_HERRAMIENTAS where jugador = ?',(jugador,))
  # fechone() / fetchall() para recuperar los resultados
  registro = cursor.fetchall() # retorna tupla con los valores de las columnas
  # con los parámetros cursor y comprobado el nombre.
  return len(registro)!= 0


def imprimir_herramientas_jugador(cursor):
  '''Imprimir las herramientas de un jugador

  Args
  ---
```

```
  cursor (cursor)
    Objeto de la BDA que permite ejercutar instrucciones SQL y recuperar registros
de la BDA.

  Returns:
    No hay. Se muestra por consola las herramientas de un jugador introducido.
  '''
  print("HERRAMIENTAS JUGADOR")
  nombre = input ('Indica el nombre del jugador: ').lower().strip()
  if not existe_jugador(cursor, nombre.lower()):
    print(f'\u001B[33mWARNING: el/la jugador/a {nombre} no existe.\u001B[0m')
    return

  if not existe_herramienta_asociada_jugador(cursor, nombre):
    print(f'\u001B[33mWARNING: el/la jugador/a {nombre} no tiene asociada'+
       ' niguna herramienta.\u001B[0m')
    return
  cursor.execute('SELECT * from LISTA_HERRAMIENTAS where jugador = ?',(nombre,))
  # fechone() / fetchall() para recuperar los resultados
  registro = cursor.fetchall() # retorna tupla con los valores de las columnas

  print(f'El/la jugador/a {nombre} tiene las herramientas:', end=" ")
  for reg in registro:
    # se extrae los datos por su indice para evitar la tupla entera
    if reg[0]==nombre: # para buscar las herramientas del jugador introducido
      # se extrae los datos por su indice para evitar la tupla entera
      print(f'{reg[1]}', end=' - ')


def quitar_herramienta(conexion, cursor):
  '''Borrar herramienta de la base de datos

  Args
  ---
  cursor (cursor)
    Objeto de la BDA que permite ejercutar instrucciones SQL y recuperar registros
de la BDA.
  Conexion (sqlite3.connect())

  Returns:
    No hay. Elimina de la BDA la herramienta de un jugador seleccionado por el
usuario.
  '''
  print("QUITAR HERRAMIENTA")
  nombre = input ('Indica el nombre del jugador: ').lower().strip()
  # Comprobando si el jugador se encuentra en la tabla JUGADORES
```

```
  if not existe_jugador(cursor, nombre):
      print(f'\u001B[33mWarning: el/la jugador/a {nombre} no existe.\u001B[0m')
      return # sale de la función
  # Comprobar si el jugador tiene asociada una herramienta
  if not existe_herramienta_asociada_jugador(cursor, nombre):
      print(f'\u001B[33mWARNING: el/la jugador/a {nombre} no tiene asociada'+
          ' niguna herramienta.\u001B[0m')
      return
  herramienta = input ('Indica la herramienta del jugador: ').lower().strip()
  # con los parámetros cursor y comprobado el nombre, solo falta un bucle
  # condicional para borrar la herramienta indicada por el usuario.
  delHerramienta = '''delete from LISTA_HERRAMIENTAS
        where jugador = ? and herramienta = ?'''
  if existe_herramienta_jugador(cursor, nombre, herramienta ):
    cursor.execute(delHerramienta, (nombre, herramienta))
    conexion.commit()
    print(f'\u001B[32mSe ha eliminado correctamente la herramienta {herramienta}'
       +f' del jugador {nombre}\u001B[0m')
  else:
    print(f'\u001B[33mWarning: El/la jugador/a {nombre} No tiene'
             +f' la herramienta: {herramienta}\u001B[0m')


# PROGRAMA PRINCIPAL ----------------------------------------
#se muestra la ruta de trabajo actual amb el metode getcwd() ""
print(f"Trabajando en la ruta:\n{os.getcwd()}")

ruta_fichero = input("\nIndica la ruta relativa hasta el fichero .db"
          +" (intro si es la misma): ")
# EjerciciosVoluntarios_Obligatorio\Ejer4Voluntario
nomFichero = input('\nIndica el nombre del fichero de la BD: ')
# ejers_voluntarios_ejer4.db
ruta_completa = os.path.join(os.getcwd(), ruta_fichero, nomFichero) #directorio
real de trabajo
#Se abre la conexión con la base de datos
conexion = sqlite3.connect(ruta_completa)
cursor = conexion.cursor()

opcion = valida_opcion()
while opcion != '0':
  if opcion == '1': #Imprimir jugadores
    imprimir_jugadores(cursor)
  elif opcion == '2': #Modificar vida de jugador
    modificar_vida(conexion, cursor)
  elif opcion == '3': #Insertar nuevo jugador
    insertar_jugador(conexion, cursor)
```

```
  elif opcion == '4': #Imprimir herramientas-jugador
    imprimir_herramientas_jugador(cursor)
  elif opcion == '5': #Quitar herramienta-jugador
    quitar_herramienta(conexion, cursor)
  opcion = valida_opcion()
#Se cierra la conexión con la base de datos.
conexion.close()
```

Resultado

Tras realizar pruebas del funcionamiento del código anterior se obtiene los siguientes resultados:

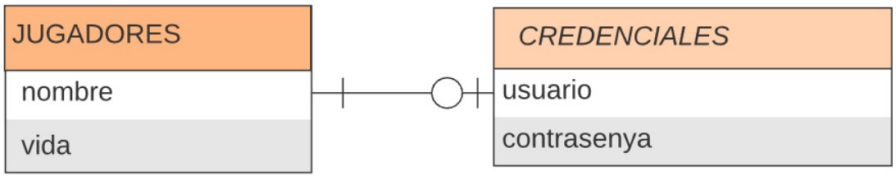


Como se puede ver en la imagen anterior, se va utilizando las funciones a partir de las opciones, de arriba a abajo y de izquierda a derecha, trabajando los máximos casos posibles indicados en la actividad como en los ejercicios anteriores.

Se han realizado cambios en relación a la última defensa, mejorando la documentación de las funciones, añadiendo alertas *Warning*, organizando mejor el contenido al imprimir por consola, se ha mejorado la experiencia de usuario (UX) a la hora de notificar errores cuando toca al igual que cuando se ha realizado correctamente, entre otras (ver en el punto problemas y soluciones).

Actividad Obligatoria

El programa a crear tiene que gestionar las credenciales de los jugadores de la base de datos ejer_obligatorio.db cuyo diagrama E-R es el siguiente:



Como se puede observar, un jugador puede no tener asociada una credencial pero si la tiene, solo tendrá una. Y una credencial pertenecerá obligatoriamente a un jugador.

El programa mostrará el siguiente menú:

```
==================================
     CREDENCIALES JUGADORES
==================================
 1 - Imprimir jugadores
 2 - Validar credencial
 3 - Cambiar credencial
 4 - Insertar nuevo jugador
 5 - Borrar jugador
 0 - SALIR
--------------------------
Dame la opción:
```

Cada opción seleccionada ejecutará una nueva función que responderá a la tarea.

Opción	Función

1	***imprimir_jugadores()***
	Esta función leerá todos los registros de la tabla JUGADORES y mostrará la información por pantalla.
	Ejemplo:
	`IMPRIMIR JUGADORES`
	`Nombre: ana - Vida: 100`
	`Nombre: pep - Vida: 120`
	`Nombre: mary - Vida: 80`
	`Nombre: quico - Vida: 100`
	`Nombre: sandy - Vida: 80`
	A la función se le pasará como parámetro de entrada el cursor a la base de

<table>
<tr><td></td><td>datos y no devolverá ningún parámetro de salida.</td></tr>
<tr><td>2</td><td>validar_credencial()

Esta función comprueba si un par usuario-contraseña existe en la tabla CREDENCIALES. Para ello, pedirá los datos por pantalla.

Ejemplo 1: La credencial existe.
<pre>VALIDAR CREDENCIAL
Dame jugador: ana
Dame contraseña: 111
[ana/111] sí es una credencial válida</pre>
Ejemplo 2: La credencial no existe.
<pre>VALIDAR CREDENCIAL
Dame jugador: ana
Dame contraseña: hola
[ana/hola] no es una credencial válida</pre>
A la función se le pasará como parámetro de entrada el cursor a la base de datos y no devolverá ningún parámetro de salida.</td></tr>
<tr><td>3</td><td>cambiar_credencial()

Esta función cambiará la contraseña de un usuario. Para ello, pedirá por pantalla el usuario, la contraseña vieja y una contraseña nueva. Si todos los datos son correctos, actualizará la tabla CREDENCIALES.

Ejemplo 1: Se cambia la credencial correctamente
<pre>CAMBIAR CREDENCIAL
Dame jugador: ana
Dame contraseña vieja: 111
Dame contraseña nueva: 999
La contraseña del jugador <ana> se ha modificado correctamente</pre>
La función deberá comprobar que el jugador existe en la base de datos y, si no existe, mostrará un mensaje de error. Para ello se creará un función nueva llamada existe_jugador() que se explica más adelante.

Ejemplo 2: El jugador no existe en la base de datos.
<pre>CAMBIAR CREDENCIAL
Dame jugador: kk
ERROR: El jugador no existe</pre>
La función deberá comprobar que la credencial introducida por teclado (usuario, contraseña) jugador existe en la base de datos y, si no existe, mostrará un mensaje de error. Para ello se creará un función nueva llamada existe_credencial() que se explica más adelante.

Ejemplo 3: El usuario existe pero la credencial introducida es incorrecta.</td></tr>
</table>

	CAMBIAR CREDENCIAL Dame jugador: ana Dame contraseña vieja: hola [ana/hola] no es una credencial válida A la función se le pasará como parámetro de entrada la conexión a la base de datos y el cursor pero no devolverá ningún parámetro de salida.
4	***insertar_jugador()*** Esta función pedirá por teclado los datos de un nuevo jugador y, si no existe, lo dará de alta en la base de datos. Para ello, deberá actualizar los datos de la tabla JUGADORES y la tabla CREDENCIALES. Ejemplo 1: Todos los datos son correctos y se da de alta el nuevo jugador. INSERTAR JUGADOR Dame nombre: perico Dame vida: 95 Dame contraseña: 666 El jugador <perico> se ha insertado correctamente Ejemplo 2: El usuario ya existía en la base de datos. INSERTAR JUGADOR Dame nombre: perico ERROR: El jugador ya existe A la función se le pasará como parámetro de entrada la conexión a la base de datos y el cursor pero no devolverá ningún parámetro de salida.
5	***borrar_jugador()*** Esta función pedirá un nombre de jugador por teclado y borrará todos los registros relacionados de las tablas JUGADORES y CREDENCIALES. Ejemplo 1: El jugador existe y se borra correctamente BORRAR JUGADOR Dame el nombre del jugador: perico Se ha eliminado el jugador <perico> correctamente Ejemplo 2: El jugador no existe BORRAR JUGADOR Dame el nombre del jugador: kk ERROR: El jugador no existe A la función se le pasará como parámetro de entrada la conexión a la base de datos y el cursor pero no devolverá ningún parámetro de salida.
0	El programa acabará.

Cuando se acabe de ejecutar una acción, el programa volverá a mostrar el menú al usuario, excepto si se ha seleccionado la opción 0.

En esta tabla se describen las funciones adicionales que debes programar:

Función	Descripción
existe_jugador()	Esta función comprobará si el jugador existe en la tabla JUGADORES pero no mostrará nada por pantalla. A la función se le pasará como parámetro de entrada el cursor de la base de datos y el nombre del jugador y devolverá como parámetro de salida un valor lógico: ***True*** si el jugador existe en la tabla JUGADORES y ***False*** en otro caso.
existe_credencial()	Esta función comprobará si existe la credencial en la tabla CREDENCIALES dado un nombre de usuario y contraseña pero no mostrará nada por pantalla. A la función se le pasará como parámetro de entrada el cursor de la base de datos y el nombre del jugador y devolverá como parámetro de salida un valor lógico: ***True*** si el jugador existe en la tabla JUGADORES y ***False*** en otro caso.

En algunos casos hará falta manejar excepciones y, en otros, solo será necesario imprimir un aviso al usuario.

Puedes crear el programa desde cero o basarte en el esquema que tienes en el fichero ud2_ejercicio_obligatorio (ESQUEMA).py del aula virtual. En este caso, deberás sustituir las instrucciones pass por las instrucciones adecuadas para que funcione el programa según el enunciado.

Importante: Para utilizar la librería pwinput, para ocultar las contraseñas y que aparezca las teclas pulsadas, hay que instalarla con *pip install pwinput*, en mi caso he tenido que realizar un *py -m pip install pwinput*

Siguiendo todos los conceptos de la unidad, se procede a trabajarlos en el código siguiente del ejercicio, siguiendo además las indicaciones del ejercicio con el fichero que contiene el esquema. A continuación se muestra todas las funciones utilizadas para alcanzar el objetivo de este ejercicio:

```
def imprimir_jugadores(cursor):
  '''Imprimir jugadores existentes en la base de datos

  Args
  ---
  cursor (cursor)
    Objeto que permite ejecutar instrucciones SQL y recuperar registros de la BDA.

  Returns:
    Out: No hay. Muestra por consola las columnas nombre y vida de la tabla
JUGADORES.
  '''
  print("IMPRIMIR JUGADORES")
  cursor.execute('SELECT nombre, vida from JUGADORES')
```

```
  # fechone() / fetchall() para recuperar los resultados
  reg = cursor.fetchone() # retorna tupla con los valores de las columnas
  c = 0 # contador
  while reg != None: # None si no hay nada que retornar
    c+=1
    print(f'jugador {c:>2}: Nombre: {reg[0]:<6} - Vidas: {reg[1]:>3}')
    reg = cursor.fetchone() # siguiente registro

def existe_jugador(cursor, jugador):
  '''Comprueba si existe un jugador en la tabla JUGADORES.

  Args
  ---
  cursor (cursor)
    Objeto de la BDA que permite ejecutar instrucciones SQL y recuperar registros
de la BDA.
  jugador (string)
    necesario para comparar con los jugadores de la BDA.

  Returns:
    Boolean: devuelve True si existe y False en otro caso.
  '''
  cursor.execute('SELECT * from JUGADORES where nombre = ?',(jugador,))
  # fechone() / fetchall() para recuperar los resultados
  registro = cursor.fetchone() # retorna tupla con los valores de las columnas
  return (registro != None)

def existe_credencial(cursor, usuario, contras):
  '''Comprueba si existe una credencial en la tabla CREDENCIALES.

  Args
  ---
  cursor (cursor)
    Objeto de la BDA que permite ejecutar instrucciones SQL y recuperar registros
de la BDA.
  usuario (string)
    necesario para poder tener acceso..
  contras (string)
      necesario para que el usuario tenga acceso.

  Returns:
    Boolean: devuelve True si existe y False en otro caso.
  '''
  cursor.execute('SELECT * from CREDENCIALES where usuario = ? '
          +'and contrasenya = ?',(usuario,contras))
  # fechone() / fetchall() para recuperar los resultados
```

```
  registro = cursor.fetchone() # retorna tupla del valor de las columnas
  return (registro != None)

def lee_vida():
  '''Pide el valor "vida" por pantalla.

  Args
  ---
    No hay.

  Returns:
    Integer: número de vida del jugador.
  '''
  correcto = False
  while not correcto:
    try:
      vida = int(input("Dame vida: "))
    except ValueError:
      print("\n\u001B[33mWARNING:Se está esperando un número entero. "
        +"Por favor, vuelve a intentarlo.\n\u001B[0m")
    else:
      correcto = True
  return vida

def validar_credencial(cursor):
  '''Comprueba si existe el par jugador-contrasenya.

  Args
  ---
  cursor (cursor)
    Objeto de la BDA que permite ejecutar instrucciones SQL y recuperar registros
de la BDA.

  Returns:
    Out: No hay. Muestra por consola si la credencial es valida o no.
  '''
  print("VALIDAR CREDENCIAL")
  nombre = input ('Indica el nombre del jugador: ').lower().strip()
  # Confirmando usuario
  if not existe_jugador(cursor, nombre):
      print(f'\u001B[33mWARNING: El/la jugador/a {nombre} no existe.\u001B[0m')
      return # Sale de la función sin devolver nada

  # Se utiliza la libreria pwinput para ocultar las contrasenyas
  password = pwinput.pwinput('Ahora la contraseña del jugador: ',
                mask='*')
```

```
  # Comprobando el par jugador-credenciales y evitar código duplicado
  print()
  if  existe_credencial(cursor, nombre, password):
    print(f'\u001B[32mLa contraseña introducida es correcta.\u001B[0m')
  else:
    print(f'\u001B[33mWARNING: La contraseña introducida '+
       f'NO es existe.\u001B[0m')

def cambiar_credencial(conexion, cursor):
  '''Modifica la contraseña de un jugador.

  Args
  ---
  cursor (cursor)
    Permite ejercutar instrucciones SQL y recuperar registros de la BDA.
  Conexion (sqlite3.connect())

  Returns:
    No hay. Actualiza la columna contrasenya de la tabla credenciales de la BDA.
  '''
  print("CAMBIAR CREDENCIAL")
  nombre = input('Indica el nombre del jugador: ').lower().strip()

  if not existe_jugador(cursor, nombre):
    print(f'\n\u001B[33mWARNING: El/la jugador/a {nombre} no existe.\u001B[0m')
    return # Sale de la función sin devolver nada

  pwdOld = pwinput.pwinput('Ahora indica la contraseña a cambiar: ',
              mask='*')
  if not existe_credencial(cursor, nombre, pwdOld):
    print(f'\n\u001B[33mWARNING: El contraseña introducida no es'
       +f' válida.\u001B[0m')
    return # Sale de la función sin devolver nada

  # Si supera las dos estructuras condicionales se modifica
  pwdNew = pwinput.pwinput('Por último, indica la nueva contraseña: ',
              mask='*')
  mod = 'update CREDENCIALES set contrasenya = ? where usuario = ?'
  cursor.execute(mod,(pwdNew, nombre))
  conexion.commit()
  print(f'\n\u001B[32mLa contraseña del jugador "{nombre}" se ha'
     +' modificado correctamente.\u001B[0m')

def insertar_jugador(conexion, cursor):
  '''Inserta una nueva herramienta en la base de datos.
```

```
    Args
    ---
    cursor (cursor)
      Permite ejercutar instrucciones SQL y recuperar registros de la BDA.
    Conexion (sqlite3.connect())

    Returns:
      No hay. Inserta un usuario, su vida y la contrasenya en la BDA.

    '''
    print("INSERTAR JUGADOR")
    nombre = input('Indica el nombre del jugador: ').lower().strip()

    if existe_jugador(cursor, nombre):
      print(f'\n\u001B[33mWARNING: El/la jugador/a {nombre} ya existe.\u001B[0m')
      return # Sale de la función sin devolver nada

    vida = lee_vida()

    pwd = pwinput.pwinput('\nPor último, la contraseña del nuevo jugador: ',
                mask='*')
    # insertando los datos
    ins1 = 'insert into JUGADORES(nombre, vida) values(?, ?)'
    ins2 = 'insert into CREDENCIALES(usuario, contrasenya) values(?, ?)'
    cursor.execute(ins1,(nombre, vida))
    cursor.execute(ins2, (nombre, pwd))
    conexion.commit()
    print(f'\n\u001B[32mEl nuevo jugador "{nombre}" se ha insertado '+
       f'correctamente.\u001B[0m')

def borrar_jugador(conexion, cursor):
    '''Borra el jugador y sus credenciales de la base de datos

    Args
    ---
    cursor (cursor)
      Permite ejercutar instrucciones SQL y recuperar registros de la BDA.
    Conexion (sqlite3.connect())

    Returns:
      No hay. Elimina un usuario introducido de la BDA.

    '''
    print("BORRAR JUGADOR")
    nombre = input('Indica el nombre del jugador: ').lower()
```

```
  if not existe_jugador(cursor, nombre):
    print(f'\n\u001B[33mWARNING: El/la jugador/a {nombre} NO existe.\u001B[0m')
    return # Sale de la función sin devolver nada
  del1 = 'delete from JUGADORES where nombre = ?'
  del2 = 'delete from CREDENCIALES where usuario = ?'
  cursor.execute(del1, (nombre,))
  cursor.execute(del2, (nombre,))
  conexion.commit()
  print(f'\n\u001B[32mEl jugador "{nombre}" se ha borrado correctamente.\u001B[0m')

# PROGRAMA PRINCIPAL ------------------------------------------
#Se crea la conexión a la base de datos
# -------------------------------------------------------------
#se muestra la ruta de trabajo actual amb el metode getcwd() ""
print(f"Trabajando en la ruta:\n{os.getcwd()}")

ruta_fichero = input("\nIndica la ruta relativa hasta el fichero .db"
           +" (intro si es la misma): ")
# EjerciciosVoluntarios_Obligatorio\EjercicioObligatorio
nomFichero = input('\nIndica el nombre del fichero de la BD: ')
# ejer_obligatorio.db
ruta_completa = os.path.join(os.getcwd(), ruta_fichero, nomFichero) #directorio
real de trabajo
#Se abre la conexión con la base de datos
conexion = sqlite3.connect(ruta_completa)
# -------------------------------------------------------------
cursor = conexion.cursor() #Creamos el cursor

opcion = valida_opcion()
while opcion != '0':
  if opcion == '1': #Imprimir jugadores
    imprimir_jugadores(cursor)
  elif opcion == '2': #Validar credencial
    validar_credencial(cursor)
  elif opcion == '3': #Cambiar credencial
    cambiar_credencial(conexion, cursor)
  elif opcion == '4': #Insertar nuevo jugador
    insertar_jugador(conexion, cursor)
  elif opcion == '5': #Borrar jugador
    borrar_jugador(conexion, cursor)
  opcion = valida_opcion()
#Se cierra la conexión con la base de datos.
conexion.close()
```

Resultado



Como se puede ver en la imagen anterior, se va utilizando las funciones a partir de las opciones, de arriba a abajo y de izquierda a derecha, trabajando los máximos casos posibles indicados en la actividad como en los ejercicios anteriores. Por otro lado se han realizado varios cambios relacionados con la última defensa, se ha documentado las funciones, se ha utilizado la librería pwinput para las contraseñas, se ha encontrados pequeños errores y algunas cosas más.

Problemas encontrados, soluciones y/o sugerencias

Se ha encontrado un problema en el ejercicio voluntario 4, en el apartado imprimir herramientas-jugador (opcion 4). En este caso se ha detectado un problema al añadir un jugador (opción 3), que añade un jugador en la tabla JUGADORES pero no en LISTA_HERRAMIENTAS. De esta manera se pierde la relación:

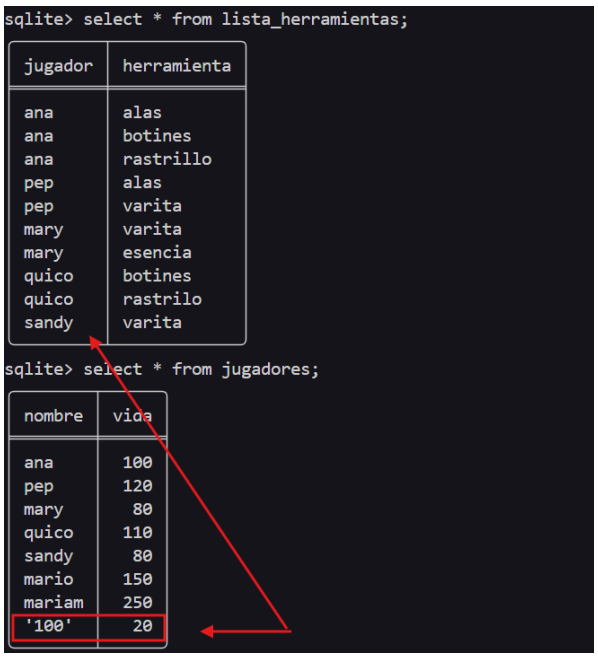


Figura 1: Relación entre las tablas JUGADORES y LISTA_HERRAMIENTAS

En mi caso, aparecía la siguiente confusión:

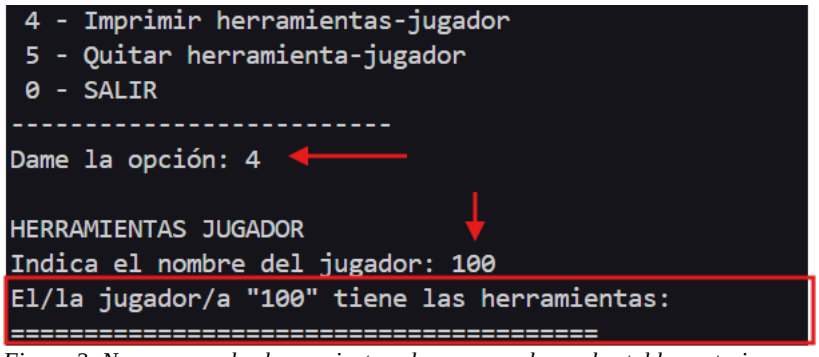


Figura 2: No aparecen las herramientas al no generarlas en las tablas anteriores.

Para solucionar estos pequeños errores se utiliza una nueva función llamada existe_herramienta_asociada_jugador().

```
 4 - Imprimir herramientas-jugador
 5 - Quitar herramienta-jugador
 0 - SALIR
--------------------------
Dame la opción: 4

HERRAMIENTAS JUGADOR
Indica el nombre del jugador: 100
WARNING: el/la jugador/a 100 no tiene asociada niguna herramienta.
```

Figura 3: Solucionado

Conclusión

Tras la última defensa he visto una oportunidad de conseguir aplicar los conocimientos que tanto de la documentación como del profesor, aportando mucho conocimiento que me han ayudado mucho más a trabajar tanto el aspecto de la consola como la experiencia que puede crearle al usuario. En resumen, con ganas de más.

Bibliografía/webgrafía

- Documentación del curso.
- Repositorio de GitHub compartido: https://github.com/oscgaresc/ADA